洋葱 Onion · 总施工说明书

> **这是给 Code Agent 的单一入口。** 读完这一份即可开工，不需要翻其他文档。

> 附录索引在最后一节，其余六份文档是**展开细节**，需要时再查。

---

0. 三十秒速览

项	内容
产品	本地优先的 AI 知识工具。点开任意知识点像洋葱一样层层剥开；AI 会像向导一样带着你问下去
形态	浏览器 Web App（PWA），无后端。用户自配模型 API Key，数据全存本机 IndexedDB
画布	**React Flow + ELK** 无限画布，卡片流，悬停高亮祖先路径
差异化	七视角内容切换 + 掌握度追踪 + SM-2 复习 + 缺口检测 + 10 家模型自配 + 纯本地
工期	**15 个工作日**（1 名全职 Code Agent）
目录代号	`onion`

---

1. 产品要解决的真实问题

用户在某个知识点上"卡住"时，会用 AI 问一句、得到一段文字、然后不知道自己接下来该问什么。反复几轮就累了，而且聊完的东西全部消失。

洋葱要做三件事：

1. **结构不跑偏** —— 每次展开都遵循固定的六层语义骨架（本质 / 构成 / 机理 / 边界 / 关联 / 延伸），不是每次换个角度重新发挥

2. **有路可走** —— 面对空白画布也知道第一步点哪（新节点在旁生长、追问气泡随时可开）

3. **不丢** —— 每一次出题、每一处标记、每一个没走到的分支，都沉淀进知识树并主动推回面前

1.1 闭环（验收硬标准）

```
[1] 播种 → [2] 剥层 → [3] 钻取 → [4] 检验 → [5] 复习 → [6] 重组 → [7] 再播种
```

三条**必须实现**的回灌链路：

#	链路	验收方式
1	答错题 → 该节点 `needsExpand=true` → 卡片显示橙色「待补全」徽标	点错一题，徽标必现
2	连续复习失败 ≥2 次 → 该节点及其子树重新开放生成	连续答错两题，节点重开
3	缺口检测 → 命中结构缺陷的节点进入「待补全清单」	人工用例 100% 命中

---

2. 核心机制：双轴洋葱模型

2.1 轴一：六层语义骨架（管结构）

每一层的语义目标固定不变，AI 永远按这张表生成子节点。这是"知识树不长歪"的唯一保证。

层	名称	语义目标	子节点数
L0	本质	这个概念到底是什么，属于什么范畴	节点自身
L1	构成	它由哪几块组成，每块是什么	3–6
L2	机理	它内部怎么跑，因果链是什么	3–6
L3	边界	什么条件下不成立，反例与误解	3–6
L4	关联	它和别的东西什么关系	3–6
L5	延伸	还能往哪走，变体与争议	3–6

硬约束

- 深度上限 L5，L5 的子节点收纳进「延伸」桶，不再生成 L6
- 每次 API 调用**只生成一层**（禁止多层，会导致幻觉与结构失控）
- 子节点 3–6 个，去重（与兄弟 + 祖先比较）
- 层色全局恒定：L1 靛蓝 / L2 青 / L3 琥珀 / L4 紫 / L5 品红

2.2 轴二：七视角内容切换（管内容）

每个节点内部有 7 个可切换视角，决定"展开后看到什么"。层管结构，视角管内容。

key	中文	内容
`overview`	速览	三句话：是什么 / 为什么重要 / 一句话本质
`mechanism`	原理	详细解释、步骤、公式、推导
`evidence`	实证	案例、数据、实验、文献
`application`	应用	使用场景、实操步骤、避坑
`controversy`	争议	不同学派观点、局限性、批评
`relation`	关系	上游前置、下游后继、易混概念
`ask`	追问	自由提问框，答案可固化为新节点

2.3 展开引擎契约

```
// src/engine/expandEngine.ts
export interface ExpandContext {
  parentNode: OnionNode;
  targetLayer: LayerIndex;          // 1..5
  siblingTitles: string[];          // 去重用
  ancestorPath: string[];           // 面包屑
  pathContext?: string;             // 仅 L4 关联层注入
}
export interface ExpandResult {
  children: Array<{ title: string; essence: string; detail: string }>;
  usage: ChatUsage;
  cached: boolean;
}
export async function expand(
  ctx: ExpandContext, config: ProviderConfig,
  settings: AppSettings, signal?: AbortSignal,
): Promise<ExpandResult>;
```

流程

1. 校验 `targetLayer ≤ 5`，否则拒绝并提示"已到最外层"

2. 组装 prompt（§5.1）

3. `provider.chat({ messages, temperature: 0.6, json: true })`

4. `parseLooseJSON` 三级容错解析（§5.3）

5. 校验：数量 3–6、标题去重、essence 长度 15–40 字

6. 失败重试 2 次（退避 1s / 3s），仍失败 → 降级为「手动添加子节点」入口，**绝不静默失败**

---

3. 技术栈（锁定，不要改）

```
{
  "dependencies": {
    "react": "^18.3.1", "react-dom": "^18.3.1",
    "zustand": "^4.5.5", "dexie": "^4.0.10",
    "@xyflow/react": "^12.3.6", "elkjs": "^0.9.3",
    "marked": "^14.1.2", "dompurify": "^3.2.3", "katex": "^0.16.11",
    "date-fns": "^4.1.0", "clsx": "^2.1.1"
  },
  "devDependencies": {
    "vite": "^5.4.11", "@vitejs/plugin-react": "^4.3.4",
    "typescript": "^5.6.3", "tailwindcss": "^3.4.15",
    "postcss": "^8.4.49", "autoprefixer": "^10.4.20",
    "vitest": "^2.1.8", "@testing-library/react": "^16.0.1",
    "@testing-library/jest-dom": "^6.6.3", "jsdom": "^25.0.1",
    "@playwright/test": "^1.49.1", "vite-plugin-pwa": "^0.21.1"
  }
}
```

明确不使用：d3-hierarchy、zustand-persist、任何 UI 组件库、任何 LLM SDK、任何后端。

> **加依赖前必须说明理由并获得确认。**

3.1 初始化

```
npm create vite@latest onion -- --template react-ts
cd onion
npm i zustand dexie @xyflow/react elkjs marked dompurify katex date-fns clsx
npm i -D tailwindcss@3 postcss autoprefixer vitest @testing-library/react \
  @testing-library/jest-dom jsdom @playwright/test vite-plugin-pwa
npx tailwindcss init -p
```

`tailwind.config.js`：

```
export default {
  content: ['./index.html', './src/**/*.{ts,tsx}'],
  darkMode: ['class', '[data-theme="dark"]'],
  theme: { extend: { colors: { /* 见 §6 令牌映射 */ } } },
  corePlugins: { preflight: false },   // 保留浏览器默认样式，避免与令牌冲突
}
```

3.2 目录结构（照此创建）

```
src/
├─ domain/                  # 纯逻辑，零 UI 零 IO，必须有单测
│  ├─ types.ts              # §4 全量类型
│  ├─ layers.ts             # 六层定义 + prompt 模板 + 校验
│  ├─ facets.ts             # 七视角定义
│  ├─ tree.ts               # 增删改查、路径计算、移动、环检测
│  ├─ mastery.ts            # 掌握度三态 + 计算
│  ├─ srs.ts                # SM-2 间隔重复
│  ├─ gapDetection.ts       # 五条缺口规则
│  ├─ pathHighlight.ts      # 祖先路径高亮算法（§7.3）
│  └─ elkLayout.ts          # ELK 布局封装
├─ engine/
│  ├─ providers/
│  │  ├─ types.ts  base.ts  openai.ts  anthropic.ts
│  │  ├─ qwen.ts  zhipu.ts  deepseek.ts  kimi.ts
│  │  ├─ doubao.ts  ollama.ts  openrouter.ts  registry.ts
│  ├─ prompts/  expand.ts  facets.ts  quiz.ts  bridge.ts
│  ├─ expandEngine.ts  quizEngine.ts  json.ts
├─ db/          schema.ts  repository.ts
├─ store/       useTreeStore.ts  useUIStore.ts  useSettingsStore.ts
├─ components/
│  ├─ canvas/   OnionCanvas.tsx  CardNode.tsx  FollowUpNode.tsx  MiniMap.tsx  AskBubble.tsx
│  ├─ detail/   FacetTabs.tsx  MarkdownView.tsx  QuizPanel.tsx  AskPanel.tsx
│  ├─ sidebar/  TreeList.tsx  SearchBox.tsx  ReviewQueue.tsx  GapList.tsx
│  ├─ settings/ ProviderCard.tsx  ProviderForm.tsx  ModelPicker.tsx  KeyInput.tsx
│  └─ common/   Toast.tsx  ConfirmDialog.tsx  EmptyState.tsx
├─ pages/       HomePage.tsx  CanvasPage.tsx
├─ lib/         crypto.ts  storage.ts  download.ts  clipboard.ts  markdown.ts
└─ styles/      tokens.css  global.css
tests/e2e/*.spec.ts
public/brand/*.svg       # 从方案包 brand/ 复制
```

---

4. 数据模型（完整类型定义）

```ts
// src/domain/types.ts
export type LayerIndex = 0 | 1 | 2 | 3 | 4 | 5;
export type FacetKey =
  | 'overview' | 'mechanism' | 'evidence'
  | 'application' | 'controversy' | 'relation';
export type MasteryState = 'mastered' | 'fuzzy' | 'unknown';
export type NodeSource = 'user' | 'ai' | 'import';
export type EdgeType =
  | 'prereq' | 'derive' | 'contain' | 'contrast'
  | 'analogy' | 'apply' | 'cause';

export interface OnionNode {
  id: string;
  treeId: string;
  parentId: string | null;
  layer: LayerIndex;
  title: string;
  essence: string;                          // 一句话本质，15–40 字
  facets: Partial<Record<FacetKey, string>>; // Markdown，未生成的不存在
  score: number;                            // 0..100
  mastery: MasteryState;
  needsExpand: boolean;
  verified: boolean;
  tags: string[];
  source: { type: NodeSource; note?: string; model?: string; at?: number };
  qa: Array<{ q: string; a: string; at: number }>;
  order: number;
  createdAt: number;
  updatedAt: number;
  lastReviewedAt?: number;
  reviewCount: number;
  lapses: number;
}

export interface OnionTree {
  id: string; name: string; rootId: string; nodeIds: string[];
  createdAt: number; updatedAt: number;
  reviewSettings: { enabled: boolean; dailyLimit: number };
}

export interface OnionEdge {
  id: string; fromId: string; toId: string;
  type: EdgeType; note?: string; weight: number; createdAt: number;
}

export interface ReviewRecord {
  id: string; nodeId: string;
  ease: number;         // SM-2 ease factor
  interval: number;     // 天
  dueAt: number; lastAt: number; quality: number;
  repetitions: number; lapses: number;
}

export type QuestionType = 'choice' | 'judge' | 'short';

export interface QuizQuestion {
  type: QuestionType;
  question: string;
  options?: string[];                       // choice 专有，4 项
  answer: number | string | boolean;        // choice/judge 为索引；short 为要点串
  explanation: string;
  difficulty: 'easy' | 'medium' | 'hard';
  sourceNodeIds: string[];
}
```

```ts
export interface QuizSession {
  id: string; nodeId: string;
  questions: QuizQuestion[];
  answers: (number | string | boolean | null)[];
  correct: number; total: number;
  finished: boolean; createdAt: number; completedAt?: number;
}

export interface GapItem {
  nodeId: string; treeId: string; reason: string;
  severity: 'high' | 'mid' | 'low';
  detectedAt: number; resolved: boolean;
}

export type ProviderId =
  | 'openai' | 'anthropic' | 'qwen' | 'zhipu' | 'deepseek'
  | 'kimi' | 'doubao' | 'openrouter' | 'ollama' | 'custom';

export interface ProviderConfig {
  id: string; providerId: ProviderId;
  baseUrl?: string;
  apiKey: string;                          // 运行时内存
  apiKeyEnc?: string;                      // 持久化密文
  rememberKey: boolean;
  model: string;
  headers?: Record<string, string>;
  enabled: boolean;
  lastTestedAt?: number; lastLatency?: number; lastError?: string;
  createdAt: number;
}

export interface AppSettings {
  theme: 'dark' | 'light';
  activeConfigId: string | null;
  smartRouting: {
    enabled: boolean;
    fastConfigId: string | null;           // 展开、生成速览用
    strongConfigId: string | null;         // 出题、对比、桥接用
  };
  thinkingEnabled: boolean;                // 全局思考模式开关
  temperature: number; maxTokens: number;
  cacheTtlHours: number;
  childCountRange: [number, number];
  language: 'zh-CN' | 'en-US';
}

export interface CacheEntry {
  key: string;                             // hash(provider+model+prompt)
  response: string; createdAt: number; expiresAt: number;
}

export interface ChatMessage {
  id: string; nodeId: string;
  role: 'user' | 'assistant';
  content: string; createdAt: number; tokens?: number;
}
```

4.1 Dexie Schema

```
// src/db/schema.ts
import Dexie, { Table } from 'dexie';

export class OnionDB extends Dexie {
  nodes!: Table<OnionNode, string>;
  trees!: Table<OnionTree, string>;
  edges!: Table<OnionEdge, string>;
  reviews!: Table<ReviewRecord, string>;
  quizzes!: Table<QuizSession, string>;
  gaps!: Table<GapItem, string>;
  providers!: Table<ProviderConfig, string>;
  settings!: Table<{ key: string; value: unknown }, string>;
  cache!: Table<CacheEntry, string>;
  chats!: Table<ChatMessage, string>;

  constructor() {
    super('onion');
    this.version(1).stores({
      nodes: 'id, treeId, parentId, [treeId+layer], score, mastery, needsExpand, updatedAt',
      trees: 'id, name, updatedAt',
      edges: 'id, fromId, toId, [fromId+toId], type',
      reviews: 'id, nodeId, dueAt',
      quizzes: 'id, nodeId, createdAt',
      gaps: 'id, nodeId, treeId, resolved, severity',
      providers: 'id, providerId, enabled',
      settings: 'key',
      cache: 'key, expiresAt',
      chats: 'id, nodeId, createdAt',
    });
  }
}
export const db = new OnionDB();
```

> ⚠️ **Dexie 不支持 boolean 索引**。`needsExpand` 存储层用 `0 | 1`，读取时转回 boolean。

4.2 存储位置与加密

数据	位置	加密
树 / 节点 / 边 / 掌握度	IndexedDB `onion-*`	❌
API Key	IndexedDB `onion-settings`	✅ AES-GCM（WebCrypto，PBKDF2 210k 派生）
对话历史 / 复习 / 缺口 / 缓存	IndexedDB	❌
偏好设置	localStorage	❌

隐私铁律：除「向用户自己配置的模型厂商发请求」外，**应用不向任何自有服务器发送数据**。不埋点、不上传。

---

5. AI 引擎

5.1 Provider 抽象

```ts
// src/engine/providers/types.ts
export interface ChatMessage { role: 'system'|'user'|'assistant'; content: string; }

export interface ChatRequest {
  messages: ChatMessage[];
  model: string;
  temperature?: number;   // 默认 0.6
  maxTokens?: number;     // 默认 2048
  topP?: number;          // 默认 0.95
  stream?: boolean;       // 默认 true
  json?: boolean;         // 要求 JSON 输出
  stop?: string[];
  thinking?: boolean;     // 思考模式
  signal?: AbortSignal;
}

export interface ChatUsage { promptTokens: number; completionTokens: number; totalTokens: number; }
export interface ChatChunk { delta?: string; reasoning?: string; usage?: ChatUsage; done: boolean; }
export interface ChatResult { content: string; reasoning?: string; usage: ChatUsage; model: string; raw?: unknown; }

export interface ProviderModel {
  id: string; label: string; recommended?: boolean;
  contextWindow?: number; supportsStream?: boolean; supportsThinking?: boolean;
}

export interface LLMProvider {
  readonly id: ProviderId;
  readonly label: string;
  readonly defaultBaseUrl: string;
  readonly defaultModels: ProviderModel[];
  readonly extraHeaders?: Record<string, string>;
  readonly corsCompatible: boolean;

  chat(req: ChatRequest, apiKey: string, baseUrl?: string): Promise<ChatResult>;
  chatStream(req: ChatRequest, apiKey: string, baseUrl?: string): AsyncIterable<ChatChunk>;
  testConnection(apiKey: string, baseUrl?: string)
    : Promise<{ ok: boolean; latencyMs: number; message: string }>;
}
```

5.2 厂商清单（端点已于 2026-10 核对官方文档）

Provider	默认 BaseUrl	协议类	内置模型	特殊处理
OpenAI	`https://api.openai.com/v1`	OpenAI	`gpt-4o` `gpt-4o-mini` `gpt-4.1` `o4-mini`	浏览器直连策略有变动，需提示
Anthropic	`https://api.anthropic.com/v1`	**原生**	`claude-sonnet-4-5` `claude-opus-4-1` `claude-haiku-4-5`	**必须加 `anthropic-dangerous-direct-browser-access: true`**
通义千问	`https://dashscope.aliyuncs.com/compatible-mode/v1`	OpenAI	`qwen3.8-max` `qwen-plus` `qwen-turbo` `qwen3.5-flash`	`qwen3.5+` 默认开思考，需 `enable_thinking:false`
智谱 GLM	`https://open.bigmodel.cn/api/paas/v4`	OpenAI	`glm-5.3` `glm-5.2` `glm-4.5`	baseUrl 末尾斜杠可省；支持 `thinking:{type:'enabled'}`
DeepSeek	`https://api.deepseek.com`	OpenAI	`deepseek-chat` `deepseek-reasoner`	reasoner 必须剔除 temperature/top_p
Kimi	`https://api.moonshot.cn/v1`	OpenAI	`kimi-k2.5` `kimi-k2-turbo-preview` `moonshot-v1-128k`	
豆包	`https://ark.cn-beijing.volces.com/api/v3`	OpenAI	`doubao-seed-1-6`	**模型栏填推理接入点 ID（ep- 开头）**，不是模型名
OpenRouter	`https://openrouter.ai/api/v1`	OpenAI	`anthropic/claude-sonnet-4.5` `openai/gpt-4o` …	可选 `HTTP-Referer` / `X-Title`
Ollama 本地	`http://localhost:11434/v1`	OpenAI	`qwen3:8b` `llama3.1:8b`	需用户自行 `ollama pull`
自定义	用户填写	OpenAI	用户填写	完整暴露 baseUrl / model / headers

架构要点：8 家兼容 OpenAI 协议 → 一个 `OpenAICompatibleProvider` 基类覆盖；**只有 Anthropic 需要独立适配器**。这是代码量最小的正确做法。

5.3 Anthropic 适配器的 5 处差异（最容易错）

差异点	OpenAI	Anthropic
端点	`/chat/completions`	`/messages`
鉴权头	`Authorization: Bearer`	`x-api-key`
版本头	无	`anthropic-version: 2023-06-01`
system 消息	在 messages 数组内	**独立的顶层 `system` 字段**
响应取文本	`choices[0].message.content`	`content[]` 数组拼接 `type==='text'`
流式事件	`data: {choices[0].delta.content}`	`event: content_block_delta` + `data: {delta:{text}}`
max_tokens	可选	**必填**

5.4 Prompt 模板全集

① 展开（按层） —— `src/engine/prompts/expand.ts`

```
export const LAYER_DEFS = [
  { layer: 1, name: '构成', goal: '它由哪几个关键要素/子概念构成，每个要素各自是什么、彼此什么关系' },
  { layer: 2, name: '机理', goal: '它内部是如何运作的，给出因果链、关键步骤、涉及的核心机制' },
  { layer: 3, name: '边界', goal: '它在什么条件下会失效、有哪些反例、常见的误解是什么' },
  { layer: 4, name: '关联', goal: '它的前置知识是什么、后继通向什么、与哪些概念容易混淆' },
  { layer: 5, name: '延伸', goal: '它的进阶方向、主流变体、开放争议、典型应用场景' },
] as const;

export function buildExpandPrompt(ctx: {
  parentTitle: string; parentEssence: string;
  layerDef: { layer: number; name: string; goal: string };
  siblingTitles: string[]; ancestorPath: string[]; pathContext?: string;
}): ChatMessage[] {
  const { parentTitle, parentEssence, layerDef, siblingTitles, ancestorPath, pathContext } = ctx;

  const system = `你是一个严谨的知识结构化引擎。你的任务是把一个知识点按指定语义层次拆解为若干互不重复的子概念。

硬性要求：
1. 只输出 JSON，不要任何解释性文字或 Markdown 代码块标记。
2. 输出格式：{"children":[{"title":"子概念名","essence":"一句话本质，15-40字","detail":"2-3句说明"}]}
3. 子概念数量为 3 到 6 个。
4. 每个子概念必须与其余子概念、以及"已有概念列表"完全无重叠。
5. essence 必须能独立成立，不允许出现"它""这个"等指代。
6. 如果你对某个方面不确定，宁可少生成一个，也不要编造。
7. 全部使用简体中文。`;

  const user = `知识点：${parentTitle}
一句话本质：${parentEssence}

当前要展开的层次：**第 ${layerDef.layer} 层 · ${layerDef.name}**
本层语义目标：${layerDef.goal}

${siblingTitles.length ? `已有概念列表（不得重复）：\n${siblingTitles.map(t => `- ${t}`).join('\n')}\n` : ''}${ancestorPa
请生成第 ${layerDef.layer} 层（${layerDef.name}）的子概念，严格按上述 JSON 格式输出。`;

  return [{ role: 'system', content: system }, { role: 'user', content: user }];
}
```

② 七视角 —— `src/engine/prompts/facets.ts`

```
export const FACET_DEFS = [
  { key: 'overview',    name: '速览', instruction: '用三句话说明：这是什么 / 为什么重要 / 一句话抓住本质' },
  { key: 'mechanism',   name: '原理', instruction: '详细解释其内部机制，必要时给出步骤、公式与推导思路' },
  { key: 'evidence',    name: '实证', instruction: '给出具体案例、数据、实验或经典文献佐证，说明证据强度' },
  { key: 'application', name: '应用', instruction: '说明典型使用场景、实操步骤、常见坑与最佳实践' },
  { key: 'controversy', name: '争议', instruction: '列出不同学派/视角的观点、该概念的局限性与批评意见' },
  { key: 'relation',    name: '关系', instruction: '梳理其上游前置、下游后继、易混概念与跨领域连接' },
] as const;
```

③ 出题 —— `src/engine/prompts/quiz.ts`

```
export function buildQuizPrompt(nodeTitle: string, essence: string, subPoints: string[], count = 4) {
  return [
    { role: 'system', content: `你是命题老师。只输出 JSON，不要其他文字。
输出格式：
{"questions":[{"type":"choice|judge|short","question":"题干","options":["A项","B项","C项","D项"],"answer":"choice填正确
choice 必须有且仅有 4 个选项；judge 不要 options 字段；short 的 answer 用 ";" 分隔的要点串。` },
    { role: 'user', content: `概念：${nodeTitle}
本质：${essence}
关键要点：
${subPoints.map(p => `- ${p}`).join('\n')}

生成 ${count} 道题，难度分布：1 简单 + 2 中等 + 1 困难。题目必须能仅凭"关键要点"作答，避免超出给定范围的细节。` },
  ];
}
```

④ 桥接节点 —— `src/engine/prompts/bridge.ts`

```
export function buildBridgePrompt(a: {title:string;essence:string}, b: {title:string;essence:string}) {
  return [
    { role: 'system', content: '你是知识关联引擎。只输出 JSON。输出格式：{"bridge":{"title":"桥接概念名","essence":"一句话说明
    { role: 'user', content: `概念A：${a.title}（${a.essence}）\n概念B：${b.title}（${b.essence}）\n\n找出连接 A 与 B 的桥
  ];
}
```

5.5 JSON 容错解析（三级）

```
// src/engine/json.ts
export function parseLooseJSON<T>(raw: string): T | null {
  try { return JSON.parse(raw) as T; } catch {}                    // ① 直接解析

  const firstObj = raw.indexOf('{'), firstArr = raw.indexOf('[');
  let start = -1, end = -1;
  if (firstObj !== -1 && (firstArr === -1 || firstObj < firstArr)) {
    start = firstObj; end = raw.lastIndexOf('}');
  } else if (firstArr !== -1) {
    start = firstArr; end = raw.lastIndexOf(']');
  }
  if (start !== -1 && end > start) {                               // ② 提取最外层块
    const slice = raw.slice(start, end + 1);
    try { return JSON.parse(slice) as T; } catch {}
    try {                                                          // ③ 修复常见问题
      const fixed = slice
        .replace(/,\s*([}\]])/g, '$1')
        .replace(/(\r\n|\n|\r)/g, '')
        .replace(/,\s*"([^"]*)"\s*:\s*"([^"]*)"\s*$/, ,"$1":"$2"');
      return JSON.parse(fixed) as T;
    } catch {}
  }
  return null;
}
```

三级都失败：展示原始文本 + 提供「让 AI 重新整理」按钮，**绝不静默失败**。

5.6 错误映射与重试

情况	动作
网络错误 / 5xx	指数退避重试 2 次（1s、3s）
429 限流	退避重试（5s、15s）+ toast 提示换模型
401 / 403	**不重试**，提示「API Key 无效或无权限」
400	**不重试**，展示原始错误（多半是模型 id 不存在）
超时 >60s	提示重试 + 「换模型重试」
JSON 解析失败	重试 1 次（加强 prompt 约束），仍失败降级

Key 脱敏：`ProviderError` 构造处主动脱敏，任何日志不得出现 Key。

5.7 缓存与成本

- 响应缓存：key = hash(provider + model + systemPrompt + userPrompt + temperature)，TTL 24h，命中不发起网络请求，UI 标注「缓存」
- 每次展开前在按钮显示预估 token
- 智能分工开关：展开/速览用**便宜快速模型**，出题/对比/桥接用**强推理模型**

---

6. UI 设计令牌

6.1 暗色主题（默认）

```
:root[data-theme="dark"] {
  --bg-canvas:#0E1116; --bg-panel:#151A21; --bg-elevated:#1C232C;
  --bg-overlay:#232B36; --bg-hover:#1F2731;
  --text-primary:#E8EDF3; --text-secondary:#9AA7B5; --text-tertiary:#6B7887; --text-inverse:#0E1116;
  --border-subtle:rgba(255,255,255,.07); --border-default:rgba(255,255,255,.11); --border-strong:rgba(255,255,255,.2
  --brand:#F2A65A; --brand-hover:#FFB871; --brand-soft:rgba(242,166,90,.14);
  --layer-0:#C8D2DE; --layer-1:#7F8CF0; --layer-2:#4FBFAE;
  --layer-3:#E0A03C; --layer-4:#B583E0; --layer-5:#E06C9C;
  --mastered:#4FBFAE; --fuzzy:#E0A03C; --unknown:#E06C6C; --expanding:#F2A65A; --needs:#FF8A4C;
  --focus:rgba(242,166,90,.55);
  --shadow-md:0 4px 16px rgba(0,0,0,.22); --shadow-lg:0 16px 48px rgba(0,0,0,.36);
  --ease-out:cubic-bezier(.22,1,.36,1); --ease-spring:cubic-bezier(.34,1.56,.64,1);
}
```

6.2 亮色主题

```
:root[data-theme="light"] {
  --bg-canvas:#FBFAF7; --bg-panel:#FFFFFF; --bg-elevated:#FFFFFF;
  --bg-overlay:#FFFFFF; --bg-hover:#F4F2ED;
  --text-primary:#1A1F26; --text-secondary:#59636F; --text-tertiary:#8B94A0; --text-inverse:#FFFFFF;
  --border-subtle:rgba(20,26,34,.07); --border-default:rgba(20,26,34,.12); --border-strong:rgba(20,26,34,.22);
  --brand:#C87A28; --brand-hover:#B26A20; --brand-soft:rgba(200,122,40,.10);
  --layer-0:#4A5560; --layer-1:#4A5BD4; --layer-2:#0E8C7C;
  --layer-3:#B87A10; --layer-4:#7B4BB8; --layer-5:#C04474;
  --mastered:#0E8C7C; --fuzzy:#B87A10; --unknown:#C84C4C; --expanding:#C87A28; --needs:#D95F20;
  --focus:rgba(200,122,40,.45);
  --shadow-md:0 4px 16px rgba(20,26,34,.10); --shadow-lg:0 16px 48px rgba(20,26,34,.16);
}
```

6.3 间距 / 圆角 / 字体

```
--space-1:4px; --space-2:8px; --space-3:12px; --space-4:16px; --space-5:20px;
--space-6:24px; --space-8:32px; --space-10:40px; --space-12:48px;
--radius-sm:6px; --radius-md:10px; --radius-lg:14px; --radius-xl:20px; --radius-full:999px;
--sidebar-width:264px; --detail-width:384px; --topbar-height:52px;
--font-sans:"Inter","PingFang SC","Hiragino Sans GB","Microsoft YaHei",system-ui,sans-serif;
--font-serif:"Songti SC","Noto Serif SC",Georgia,serif;
--font-mono:"SF Mono","JetBrains Mono",Consolas,monospace;
```

字重只用 400 / 500 / 600 三档。 字号层级：display 22 / title 16 / heading 14 / body 13.5 / body-lg 15 / caption 12 / micro 11 / mono 12.5。

---

7. 画布层（核心 · 照抄已验证方案）

> 规格来自对 rabbithole.chat 探索页生产代码的逆向，已验证有效。**不要自研布局引擎。**

7.1 React Flow 配置

```
<ReactFlow
  nodes={nodes}
  edges={edges.map(e => ({
    ...e,
    style: e.data?.isHighlighted ? EDGE_HIGHLIGHT : EDGE_DEFAULT,
  }))}
  panOnScroll                              // ★ 滚轮 = 平移，不是缩放
  panOnScrollMode={PanOnScrollMode.Free}
  nodesDraggable={false}                   // ★ 位置由 ELK 决定，用户拖不乱
  fitView
  fitViewOptions={{ padding: 0.2, maxZoom: 0.8 }}
  defaultViewport={{ x: 0, y: 0, zoom: 0.4 }}
  minZoom={0.1}
  onNodeMouseEnter={highlightAncestorPath}
  onNodeMouseLeave={clearHighlight}
  defaultEdgeOptions={{ style: EDGE_DEFAULT }}
  colorMode="dark"
>
  <Background />
  <MiniMap pannable zoomable />
</ReactFlow>
```

四个决策的意义

决策	为什么这样
`panOnScroll` + `Free`	触控板双指 = 画布平移。滚轮缩放会与页面滚动冲突，是"不听话"的来源
`nodesDraggable: false`	位置由 ELK 决定，用户拖不乱结构
`fitView` + `padding: 0.2`	打开永远完整可见且留 20% 边距
`defaultViewport.zoom = 0.4`	默认看全局，主动放大看细节

7.2 ELK 布局封装

```
// src/domain/elkLayout.ts
import ELK from 'elkjs/lib/elk.bundled.js';
const elk = new ELK();

export async function layoutTree(
  nodes: OnionNode[], edges: { id: string; fromId: string; toId: string }[],
): Promise<Record<string, { x: number; y: number }>> {
  const graph = {
    id: 'root',
    layoutOptions: {
      'elk.algorithm': 'layered',
      'elk.direction': 'RIGHT',        // 从左到右生长，像思考的流向
      'elk.spacing.nodeNode': '48',
      'elk.layered.spacing.nodeNodeBetweenLayers': '80',
    },
    children: nodes.map(n => ({ id: n.id, width: 600, height: 180 })),
    edges: edges.map(e => ({ id: e.id, sources: [e.fromId], targets: [e.toId] })),
  };
  const res = await elk.layout(graph);
  const out: Record<string, { x: number; y: number }> = {};
  res.children?.forEach(c => { out[c.id] = { x: c.x!, y: c.y! }; });
  return out;
}
```

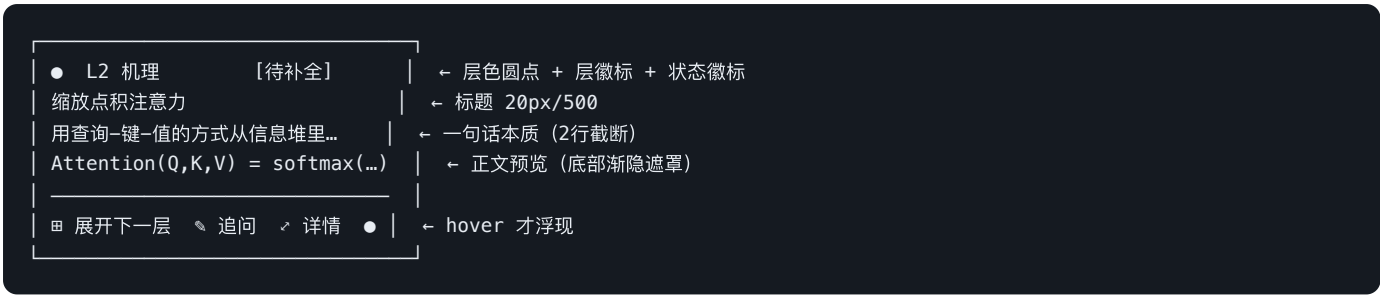
7.3 祖先路径高亮（丝滑的核心，必须实现）

```
// src/domain/pathHighlight.ts
export const EDGE_DEFAULT   = { stroke: '#374151', strokeWidth: 2, transition: 'stroke 0.2s ease' };
export const EDGE_HIGHLIGHT = { stroke: 'hsl(var(--brand))', strokeWidth: 2, transition: 'stroke 0.2s ease' };

export function getAncestorEdgeIds(
  nodeId: string,
  nodeMap: Map<string, { type: string }>,
  edges: { id: string; source: string; target: string }[],
): Set<string> {
  const ids = new Set<string>();
  let cur = nodeId;
  const node = nodeMap.get(cur);
  // 追问节点的路径起点是它引用的原节点
  if (node?.type === 'followUpNode') {
    const e = edges.find(x => x.target === cur);
    if (e) cur = e.source;
  }
  let guard = 0;
  while (cur && guard++ < 200) {
    const e = edges.find(x => x.target === cur);
    if (!e) break;
    ids.add(e.id);
    cur = e.source;
  }
  return ids;
}
```

铁律：`transition` 不可省。高亮必须是**「亮起来」而非「跳变」** —— 这是丝滑与生硬的分界线。

7.4 卡片节点规格



元素	规格
卡片宽度	`max-w-[600px]`，`minWidth: 600px`
圆角	`rounded-[35px]` —— 超大圆角是「柔和」观感的关键
内边距	`p-[30px]`
底色	`bg-[#262626]`（暗）/ `--bg-elevated`（亮）
标题	`text-[20px] font-medium`
层徽标	`px-2 py-0.5 bg-white/10 rounded-full text-xs` —— 显示 "L2 机理"
正文预览	限高 + `linear-gradient(transparent, bg)` 底部遮罩
动作区	`opacity-0 group-hover:opacity-100 transition-opacity duration-300`
状态点	右下角小圆点，实心/半透明/空心对应已掌握/模糊/未懂

7.5 追问气泡

项	值
宽度	`minWidth: 450px`
圆角 / 内边距	`rounded-[35px]` / `p-[30px]`
底色	`bg-[#1a1a1a]` + `border border-[#333]`
文本域	`bg-[#2a2a2a]` `rounded-2xl` `min-h-[130px]` `resize-none`
按钮组	`grid grid-cols-2 gap-3`，Ask 为主按钮

三个必须实现的细节

1. **挂载即聚焦** —— `useEffect(() => ref.current?.focus(), [])`，打开就能打字
2. **高度自适应** —— `el.style.height='auto'; el.style.height=el.scrollHeight+'px'`，永不出现内部滚动条
3. **Enter 提交 / Esc 取消**

7.6 转场时长基准

场景	时长	缓动
首页 → 画布	700ms	`ease-out`
卡片 hover 浮现	300ms	`ease-in-out`
边高亮	200ms	`ease`
弹窗开合	200ms	`ease`

规则：状态切换一律有过渡。工具类应用的动画不得超过 700ms。

---

8. 关键算法

8.1 掌握度模型

```
初始生成          score = 40
答对 1 题         score += 12   (上限 100)
答错 1 题         score -= 18   (下限 0)，且 needsExpand = true
标记 mastered     score = max(score, 92)
标记 fuzzy        score = min(score, 55)
标记 unknown      score = min(score, 25)，且 needsExpand = true

子树加权：
节点有效 score = 0.7 * 自身 score + 0.3 * 子树平均 score
```

```
// src/domain/mastery.ts
export const SCORE_DELTA = { correct: +12, wrong: -18 } as const;
export const SCORE_ON_MASTERY: Record<MasteryState, number> = {
  mastered: 92, fuzzy: 55, unknown: 25,
};
export const SCORE_INITIAL = 40;
export const MASTERY_LABEL: Record<MasteryState, string> = {
  mastered: '已掌握', fuzzy: '模糊', unknown: '未懂',
};
```

8.2 SM-2 间隔重复

```
// src/domain/srs.ts
export function scheduleReview(rec: ReviewRecord | null, quality: number, now = Date.now()): ReviewRecord {
  // quality: 0-5，<3 视为失败（lapses++，interval 重置为 1）
  // ease = max(1.3, ease + (0.1 - (5-q)*(0.08 + (5-q)*0.02)))
  // interval: 成功时 n==1 → 1，n==2 → 6，否则 round(prev * ease)
  // dueAt = now + interval * 86400000
}
export function dueReviews(records: ReviewRecord[], now = Date.now()): ReviewRecord[] {
  return records.filter(r => r.dueAt <= now).sort((a, b) => a.dueAt - b.dueAt);
}
```

8.3 缺口检测（五条规则）

```
// src/domain/gapDetection.ts
命中任一即标记 needsExpand：
R1  节点只有 1 个子节点（内容偏薄）
R2  L1 层缺失（直接从 L0 跳到 L2）
R3  节点 score < 30 且 7 天内未复习
R4  节点从未被任何题目覆盖到（orphan）
R5  同层中出现语义高度重叠的兄弟（title 相似度 > 0.8）
输出：GapItem[] 按 severity 排序
```

8.4 树操作安全

```
// src/domain/tree.ts
export function wouldCreateCycle(nodes, nodeId, newParentId): boolean {
  // 移动节点到自己的子孙下会成环 —— 必须拦截
}
export function getDescendants(nodes, id): string[];
export function getPath(nodes, id): OnionNode[];   // 根 → 当前
```

---

9. 交互与快捷键

9.1 画布

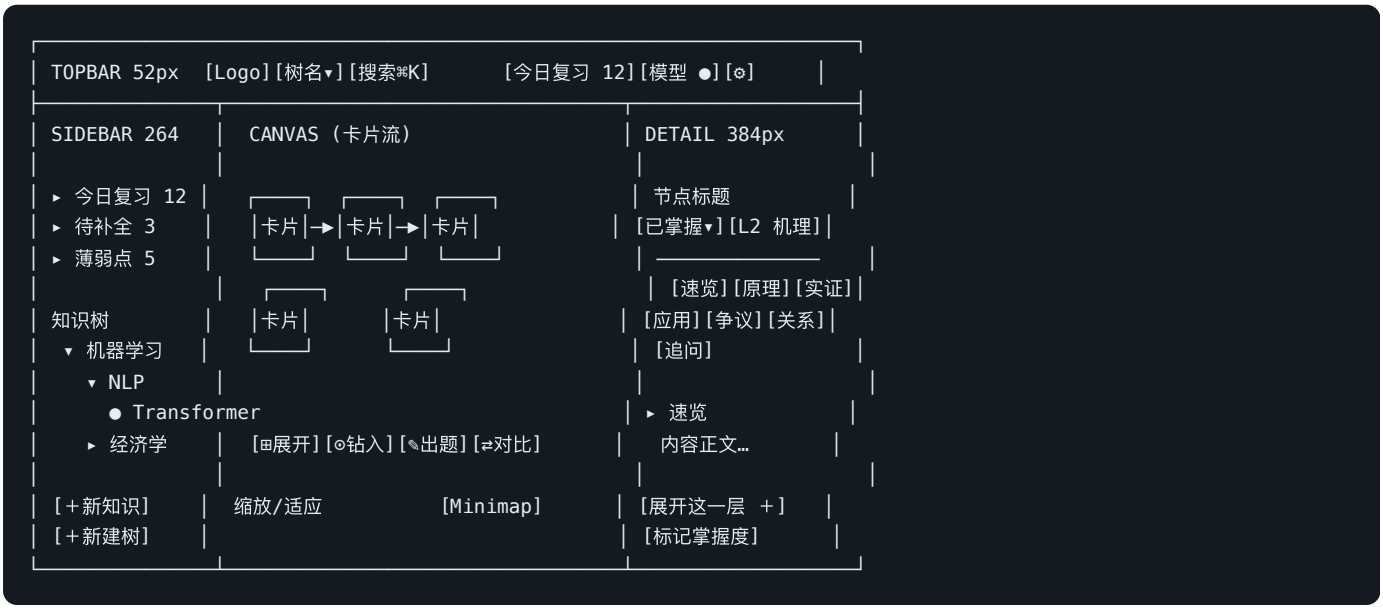
交互	行为
滚轮	平移画布
拖拽空白	平移
悬停节点	**高亮其到根节点的整条祖先路径**
悬停卡片	动作区浮现（展开下一层 / 追问 / 详情）
点击卡片	选中并同步右侧详情面板
点击「展开下一层」	生成下一层子节点，在旁生长
点击「追问」	弹出自动聚焦的输入气泡
双击节点	钻入（以此为中心重排）

9.2 快捷键

键	动作
`E`	展开选中节点
`F`	钻入选中节点
`1` – `7`	切换视角
`M`	标记掌握度三态
`Q`	出题
`⌘K`	命令面板（搜索节点 / 切换模型 / 导出 / 缺口检测）
`⌘E`	导出当前树
`⌘G`	缺口检测
`⌘,`	模型设置
`Space`	播放/暂停自动展开
`Delete`	删除选中节点（二次确认）
`Esc`	关闭弹层 / 取消选中
`↑↓`	在兄弟节点间移动
`←→`	折叠 / 展开

---

10. 布局结构



---

11. 施工任务卡（严格按序）

```
PR-1  脚手架 + 设计令牌 + 三栏布局骨架              0.5 天
PR-2  Dexie 数据层 + 类型定义 + 种子数据              1 天
PR-3  Provider 抽象 + 10 家厂商 + 设置页 + 连通性测试   2 天
PR-4  卡片流画布（React Flow + ELK + 高亮 + 气泡）    2.5 天
PR-5  展开引擎 + 六层 Prompt + 流式渲染 + 剥开动效     2.5 天
PR-6  详情面板 + 七视角 + 追问                       1.5 天
PR-7  闭环（出题判分 + 掌握度 + SM-2 + 回灌）          2 天
PR-8  缺口检测 + 导出 + 搜索 + 命令面板               1.5 天
PR-9  PWA + Logo 接入 + 打磨 + 文档                  1.5 天
                                                  合计  15 天
```

PR-1 脚手架

#	任务	验收
1.1	Vite+TS+Tailwind 打通，tokens.css 落地	亮/暗主题切换，背景与文字色随主题变化
1.2	三栏布局骨架（264 / flex / 384）	宽度符合令牌，窗口缩放不塌
1.3	Logo 接入顶栏	用 `public/brand/logo-mark.svg`，深色下清晰
1.4	顶栏 6 个组件	全部有 hover 态与真实数据绑定

PR-2 数据层

#	任务	验收
2.1	`types.ts` 全量类型	与 §4 一致
2.2	Dexie schema + 10 张表 + 索引	刷新数据不丢
2.3	仓库层 CRUD + 种子数据（3 棵树）	首次启动自动写入
2.4	`tree.ts` 单测	覆盖率 >80%，禁止把节点挂到自己的子孙下

PR-3 Provider 层

#	任务	验收
3.1	`types.ts` + `base.ts`（OpenAI 兼容基类）	基类可被 8 家复用
3.2	`anthropic.ts` 独立适配器	system 顶层字段、content 数组拼接、`x-api-key`、浏览器直连头全对
3.3	`registry.ts` 10 家配置	端点/模型与 §5.2 一致
3.4	设置页：厂商下拉分组、Key 输入、模型下拉 + 手动输入、Base URL	表单校验齐全
3.5	连通性测试	显示真实延迟，错误文案映射正确
3.6	Key 加密存储（WebCrypto AES-GCM + PBKDF2）	不勾选"记住"则刷新即失效
3.7	SSE 流式解析器	Anthropic 与 OpenAI 两种事件格式都能解析

PR-4 卡片流画布 ★

#	任务	验收
4.1	`elkLayout.ts` ELK 封装	新节点生成后自动重排，位置合理
4.2	`OnionCanvas` + `CardNode` + `FollowUpNode`	视觉符合 §7.4
4.3	滚轮平移 + 拖拽 + 缩放 + `fitView`	手感与 §7.1 参数一致
4.4	**祖先路径高亮**	悬停任一节点，其到根的路径边变品牌色
4.5	追问气泡（自动聚焦 + 高度自适应 + Enter/Esc）	三个细节全部实现
4.6	Minimap	可点击可缩放
4.7	层徽标 + 层色	L2 永远是青色，全局一致

PR-5 展开引擎

#	任务	验收
5.1	六层 Prompt 模板 + 层语义校验	与 §5.4 一致
5.2	`expandEngine.ts` 编排（去重、兄弟上下文、路径上下文）	一次调用只生成一层
5.3	`parseLooseJSON` 三级容错	注入脏数据仍能解析
5.4	流式渲染：节点占位 → 逐字上屏 → 节点长出	动画流畅
5.5	剥开动效（620ms，stagger 60ms）	观感自然
5.6	失败降级：重试 2 次 → 手动添加入口	绝不静默失败
5.7	响应缓存（prompt hash，TTL 24h）	重复展开不发网络请求

PR-6 详情面板

#	任务	验收
6.1	头部：标题 + 掌握度下拉 + 层徽标	三态可循环切换
6.2	七视角 Tab + 未生成标记点	切换流畅，内容正确
6.3	`MarkdownView`（marked + DOMPurify + KaTeX）	XSS 测试通过，公式渲染正常
6.4	追问面板 + 历史持久化	答案可固化为新节点
6.5	底部动作区	全部有真实行为

PR-7 闭环 ★

#	任务	验收
7.1	`mastery.ts` 掌握度计算	与 §8.1 一致
7.2	`quizEngine.ts` 出题 + 判分	4 题，难度 1/2/1；简答用关键词比对
7.3	测验面板 UI + 结果页	判分即时更新掌握度
7.4	`srs.ts` SM-2 调度	答对后 dueAt 正确推算
7.5	**回灌 1**：答错 → needsExpand → 橙色角标	必现
7.6	**回灌 2**：连续失败 ≥2 次 → 节点重开	必现
7.7	今日复习队列	侧栏计数准确

PR-8 增强

#	任务	验收
8.1	`gapDetection.ts` 五条规则	人工用例 100% 命中
8.2	待补全清单 + 点击直达并展开	可用
8.3	导出：单节点 / 整树 / 全库 Markdown + JSON 全量备份	JSON 导入导出 100% 无损
8.4	搜索：标题 + 内容，⌘K 唤起	模糊匹配
8.5	命令面板	§9.2 全部命令可用
8.6	节点操作：重命名 / 重新生成 / 删除 / 移动到其他树	危险操作二次确认

PR-9 交付

#	任务	验收
9.1	PWA（manifest + SW + 图标）	可安装，离线可打开
9.2	品牌资产接入（图标、启动画面、空状态线稿）	全部到位
9.3	空状态 / 错误态 / 加载态	逐条覆盖
9.4	README	新人 10 分钟跑起来
9.5	生产构建	`npm run build` 通过

---

12. 验收清单

12.1 自动化

```
npm run build                                        # 构建通过
grep -rn "TODO\|FIXME" src/ | grep -v "\.d\.ts"      # 必须无输出
npm run test -- --coverage                            # domain/ 覆盖率 >85%
npx playwright test                                   # 6 条主流程全绿
```

E2E 六条主流程

1. 配置模型 → 测试连接成功
2. 播种一个知识点 → 卡片出现在画布
3. 展开到 L5 → 每层 3–6 个子节点
4. 切换七个视角 → 内容正确渲染
5. 出题答错 → **验证橙色角标出现**
6. 刷新页面 → 数据完整保留

```
// 第 5 条断言（闭环核心验收）
await page.getByRole('button', { name: /出题/ }).click();
await page.locator('.opt').nth(0).click();
await page.getByRole('button', { name: '确认答案' }).click();
await expect(page.locator('[data-testid="needs-expand-badge"]')).toBeVisible();
```

12.2 手工走查

- [ ] 亮/暗主题切换，10 个关键页面无对比度问题
- [ ] 10 家厂商各测一次连通性（至少 5 家真实调用成功）
- [ ] 展开动画在 50+ 节点时仍流畅（60fps）
- [ ] 悬停节点 → 祖先路径高亮，移开恢复
- [ ] 追问气泡打开即聚焦，Enter 可提交
- [ ] 答错题 → 橙色角标出现 → 点击可展开 → 角标消失
- [ ] 刷新页面数据完整保留
- [ ] 导出 JSON → 清空 → 导入 → 数据完全一致
- [ ] 全程抓包确认：**无任何请求发往洋葱自有域名**
- [ ] DevTools 检查 Key 未出现在任何日志中

---

13. 常见坑（都已踩过）

坑	后果	规避
用 d3-hierarchy 做布局	强制图布局，破坏卡片流	用 elkjs
Anthropic 沿用 OpenAI 消息格式	system 提示丢失	独立适配器
Anthropic 忘记 `max_tokens`	400 报错	适配器里强制默认 2048
忘记浏览器直连头	Anthropic 全部请求 CORS 失败	`extraHeaders` 写死
千问 `qwen3.5+` 默认开思考	token 费用翻倍	`enable_thinking: false`
智谱 baseUrl 末尾斜杠	部分 SDK 报 404	`replace(/\/+$/, '')`
DeepSeek Reasoner 传 temperature	400	该模型自动剔除参数
展开时一次生成多层	结构失控、成本高	强制一层一次调用
已展开节点重复生成	出现重复子节点	生成前检查已有子节点
未过 DOMPurify	XSS	所有 AI 输出必须消毒
Key 写进 console.log	泄露	统一走脱敏工具函数
边高亮省略 transition	高亮变成"跳变"，不丝滑	`transition: stroke .2s ease` 不可省
节点允许拖动	用户拖乱结构	`nodesDraggable: false`

---

14. 品牌资产

`brand/` 目录 8 个 SVG，可直接复制到 `public/brand/`：

文件	用途
`logo-mark.svg`	主标记（深色底）
`logo-app-icon.svg`	应用图标（深色圆角底）
`logo-on-light.svg`	亮色底版本（配色加深保证对比度）
`logo-mono.svg` / `logo-mono-dark.svg`	单色版
`logo-lockup.svg` / `logo-lockup-on-light.svg`	横向组合标「洋葱 ONION」
`favicon.svg`	**简化版**（16px 可辨，必须用这个当 favicon）

Logo 构形：洋葱横切面 + 一层被剥开向外翻起。最外环留 44° 缺口正对剥片。因果自洽。

色板：洋葱皮 `#F2A65A`（暗色主题品牌色）/ `#C87A28`（亮色主题）/ 深 `#C2691F` / 浅 `#F7B267` / 核心奶油 `#FFDCA8`。

---

15. 参考原型

`prototype/index.html` —— **双击即可打开的真实交互原型**（约 116KB 单文件，零依赖）。

已实现并验证：

- 卡片流画布（左→右树状布局、悬停高亮祖先路径、滚轮平移、Minimap）
- 追问气泡（挂载即聚焦、高度自适应、Enter 提交 / Esc 取消）
- 卡片 hover 浮现动作区
- 六层语义展开（带剥开动画）

- 七视角切换 + 详情面板
- 出题判分 + 掌握度更新
- 引导串面板（分支问题串、回答决定路径、未走分支标记待补全）
- 模型设置面板（10 家厂商表单、连通性测试）
- 命令面板（⌘K）、主题切换、树侧栏联动

用法：Code Agent 可直接读取其中的 `flowLayout()` / `flowAncestorEdgeIds()` / `renderFlow()` / `openBubble()` 作为实现参考。原型的布局是简化版，React 版换 elkjs。

---

16. 附录索引

文档	内容	何时读
`00-产品方案与架构总纲.md`	产品定位、命名、双轴模型详解、闭环、里程碑	需要产品全貌时
`01-AI引擎与多模型适配规范.md`	10 家厂商完整适配器代码、Prompt 全集、错误处理	写 Provider 层时
`02-UI设计规范.md`	全部令牌、组件清单、16 个界面逐屏规格	写组件时
`03-Logo与品牌规范.md`	Logo 构形、栅格、禁例、品牌语调	接品牌资产时
`04-落地交接文档-HANDOFF.md`	任务卡明细、测试策略、常见坑	施工时对照
`05-数据模型与接口契约.md`	类型定义、Dexie schema、导入导出格式、契约变更规则	改数据结构时
`06-竞品分析与方案修订-Rabbithole.md`	Rabbithole 完整逆向分析（数据模型 + 探索页实现细节）	想理解设计来由时

版本 v1.2 | 2026-11-02 | 端点信息核对于 2026-10-30